SPARQL 1.2 Protocol

W3C Working Draft 14 August 2025

▼ **More details about this document**

This version:

https://www.w3.org/TR/2025/WD-sparql12-protocol-20250814/

Latest published version:

https://www.w3.org/TR/sparql12-protocol/

Latest editor's draft:

https://w3c.github.io/sparql-protocol/spec/

History:

https://www.w3.org/standards/history/sparql12-protocol/

Commit history

Test suite:

https://w3c.github.io/rdf-tests/

Latest Recommendation:

https://www.w3.org/TR/2013/REC-sparql11-protocol-20130321

Editors:

Andy Seaborne

Ruben Taelman

Gregory Williams

Thomas Pellissier Tanon

Former editors:

Lee Feigenbaum

Gregory Todd Williams

Kendall Grant Clark

Elias Torres

Feedback:

GitHub w3c/sparql-protocol (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[sparql12-protocol] … message topic …` (archives)

Copyright © 2008-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

The SPARQL Protocol and RDF Query Language (SPARQL) is a query language and protocol for RDF. This document specifies the SPARQL Protocol; it describes a means for conveying SPARQL queries and updates to a SPARQL processing service and returning the results via HTTP to the entity that requested them. This protocol was developed by the W3C SPARQL Working Group, part of the Semantic Web Activity as described in the activity statement .

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This specification is published by the RDF Star Working Group as part of the update of specifications for format and errata.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this specification may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 03 November 2023 W3C Process Document.

§ Set of Documents

This document is one of twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction
1.1 Document Conventions
1.2 Terminology

2. SPARQL Protocol Operations
2.1 Version Announcement
2.2 Query Operation
2.2.1 `query` via GET
2.2.2 `query` via POST with URL-encoded parameters
2.2.3 `query` via POST directly
2.2.4 Specifying an RDF Dataset
2.2.5 Accepted Response Formats
2.2.6 Success Responses
2.2.7 Failure Responses
2.3 Update Operation
2.3.1 `update` via POST with URL-encoded parameters
2.3.2 `update` via POST directly
2.3.3 Specifying an RDF Dataset
2.3.4 Success Responses
2.3.5 Failure Responses

2.4 Determining the Base IRI

3. Example SPARQL Protocol Requests (informative)

3.1 Examples of SPARQL Query

3.1.1 SELECT with service-supplied RDF Dataset

3.1.2 SELECT with simple RDF Dataset

3.1.3 CONSTRUCT with simple RDF dataset and HTTP content negotiation

3.1.4 ASK with simple RDF Dataset

3.1.5 DESCRIBE with simple RDF Dataset

3.1.6 SELECT with complex RDF Dataset

3.1.7 SELECT with query-only RDF Dataset

3.1.8 SELECT with ambiguous RDF Dataset

3.1.9 SELECT with malformed query fault

3.1.10 SELECT with query request refused fault

3.1.11 Long SELECT query using POST with URL encoding

3.1.12 Long SELECT query using direct POST

3.1.13 SELECT with internationalization

3.1.14 SELECT with reified triple patterns

3.1.15 SELECT with 1.2 query and 1.2 response using direct POST

3.2 Examples of SPARQL Update

3.2.1 UPDATE using URL-encoded parameters

3.2.2 UPDATE using POST directly

3.2.3 UPDATE specifying dataset and using POST directly

3.2.4 Multi-operation UPDATE using URL-encoded parameters

3.2.5 Multi-operation UPDATE specifying dataset and using URL-encoded parameters

3.2.6 Multi-operation UPDATE specifying dataset and using POST directly

4. Policy Considerations

4.1 Security

5. Conformance

5.1 Conformance

A. Changes between SPARQL 1.1 Protocol and SPARQL 1.2 Protocol

B. Changes between SPARQL 1.0 Protocol and SPARQL 1.1 Protocol

C. **Privacy Considerations**

D. **Security Considerations**

E. **Internationalization Considerations**

F. **Index**
F.1 Terms defined by this specification
F.2 Terms defined by reference

G. **References**
G.1 Normative references
G.2 Informative references

1. Introduction

This document describes the SPARQL 1.2 Protocol, a means of conveying SPARQL queries and updates from clients to SPARQL processors. The SPARQL Protocol has been designed for compatibility with the *SPARQL 1.2 Query Language* and with the *SPARQL 1.2 Update*. This document is primarily intended for software developers interested in implementing SPARQL query and update services and clients.

The SPARQL Protocol consists of two HTTP operations: a query operation for performing SPARQL Query Language queries and an update operation for performing SPARQL Update Language requests. SPARQL Protocol clients send HTTP requests to SPARQL Protocol services that handle the request and send HTTP responses back to the originating client.

A separate document describes the *SPARQL 1.2 Graph Store Protocol* which describes the use of HTTP operations for the purpose of managing a collection of graphs in the REST architectural style.

1.1 Document Conventions

When this document uses the words **must**, **must not**, **should**, **should not**, and **may**, and the words appear as emphasized text, they must be interpreted as described in [*RFC2119*].

§ 1.2 Terminology

SPARQL Protocol client
: An HTTP client (as defined by Section 3.3. Connections, Clients, and Servers of *HTTP Semantics* [*RFC9110*]) that sends HTTP requests for SPARQL Protocol operations. (Also known as: *client*.)

: An HTTP client (as defined by RFC 9110 [*RFC9110*]) that sends HTTP requests for SPARQL Protocol operations. (Also known as: *client*)

SPARQL Protocol service
: An HTTP server that services HTTP requests and sends back HTTP responses for SPARQL Protocol operations. The URI at which a SPARQL Protocol service listens for requests is generally known as a SPARQL endpoint. (Also known as: *service*)

SPARQL endpoint
: The URI at which a SPARQL Protocol service listens for requests from SPARQL Protocol clients.

SPARQL Protocol operation
: An HTTP request and response that conform to the protocol defined in this document.

RDF Dataset
: A collection of a default graph and zero or more named graphs, as defined by the SPARQL 1.2 Query Language.

§ 2. SPARQL Protocol Operations

The SPARQL Protocol consists of two operations: query and update. A protocol operation defines combinations of:

- The HTTP method by which the request is sent.
- The HTTP query string parameters included in the HTTP request URI.

- The message content included in the HTTP request body.
- The message content included in the HTTP response body.

The SPARQL 1.2 Protocol is built on top of HTTP. All HTTP requirements for requests and responses **must** be followed.

§ 2.1 Version Announcement

When sending a query or update operation to an endpoint, the version of this operation *MAY* be announced by either using the syntactical `version` directive, or by passing a `version` parameter (which depends on the used HTTP method).

When the version is indicated both in syntax and a parameter, they are expected to be the same. If they differ, parsers use the version from the parameter and might emit a warning about the mismatch.

To retain compability that is as broad as possible with older parsers, only queries that make use of SPARQL 1.2-specific functionality are encouraged to announce their version (i.e., for queries that do not make use of SPARQL 1.2-specific functionality it is discouraged to announce a version).

Responses to an operation *MAY* also contain version announcements. The *SPARQL results version* or *RDF version* is not necessarily the same as the *operation version* specified in the request. Instead, the version announcement is dependent on the response and its format, such as *SPARQL 1.2 Query Results JSON Format* or *RDF 1.2 Turtle*.

Values for the version string are discussed in RDF 1.2 Concepts and Abstract Syntax.

§ 2.2 Query Operation

The `query` operation is used to send a SPARQL query to a service and receive the results of the query. The query operation **must** be invoked with either the HTTP GET or HTTP POST method. Client requests for this operation **must** include exactly one SPARQL query string

(parameter name: `query`) and **may** include a version (parameter name: `version`), and zero or more default graph URIs (parameter name: `default-graph-uri`) and named graph URIs (parameter name: `named-graph-uri`). The response to a query request is either the *SPARQL 1.2 Query Results XML Format*, the *SPARQL 1.2 Query Results JSON Format*, the *SPARQL 1.2 Query Results CSV and TSV Formats*, or an RDF serialization, depending on the query form and content negotiation [*RFC9110*].

	HTTP Method	**Query String Parameters**	**Request Content Type**	**Request Message Body**
query via GET	GET	`query` (exactly 1) `version` (0 or 1) `default-graph-uri` (0 or more) `named-graph-uri` (0 or more)	None	None
query via URL-encoded POST	POST	None	`application/x-www-form-urlencoded`	URL-encoded, ampersand-separated query parameters. `query` (exactly 1) `version` (0 or 1) `default-graph-uri` (0 or more) `named-graph-uri` (0 or more)
query via POST directly	POST	`default-graph-uri` (0 or more) `named-graph-uri` (0 or more)	`application/sparql-query` (with optional `version` media-type parameter)	Unencoded SPARQL query string

The `query` request's parameters **must** be sent according to one of these three options:

§ 2.2.1 `query` via GET

Protocol clients **may** send protocol requests via the HTTP GET method. When using the GET method, clients **must** URL percent encode all parameters and include them as query parameter strings [*RFC3986*] with the names given above.

HTTP query string parameters **must** be separated with the ampersand (&) character. Clients may include the query string parameters in any order.

The HTTP request **must not** include a message body.

§ 2.2.2 `query` via POST with URL-encoded parameters

Protocol clients **may** send protocol requests via the HTTP POST method by URL encoding the parameters. When using this method, clients **must** URL percent encode [*RFC3986*] all parameters and include them as parameters within the request body via the `application/x-www-form-urlencoded` media type with the name given above. Parameters **must** be separated with the ampersand (&) character. Clients may include the parameters in any order. The content type header of the HTTP request **must** be set to `application/x-www-form-urlencoded`.

§ 2.2.3 `query` via POST directly

Protocol clients **may** send protocol requests via the HTTP POST method by including the query directly and unencoded as the HTTP request message body. When using this approach, clients **must** include the SPARQL query string, unencoded, and nothing else as the message body of the request. Clients **must** set the content type header of the HTTP request to `application/sparql-query`, with an optional `version` media-type parameter. Clients **may** include the optional `default-graph-uri` and `named-graph-uri` parameters as HTTP query string parameters in the request URI. Note that UTF-8 is the only valid charset here.

§ 2.2.4 Specifying an RDF Dataset

A SPARQL query is executed against an RDF Dataset. The RDF Dataset for a query may be specified either via the `default-graph-uri` and `named-graph-uri` parameters in the SPARQL Protocol or in the SPARQL query string using the `FROM` and `FROM NAMED` keywords. If different RDF Datasets are specified in both the protocol request and the SPARQL query string, then the SPARQL service **must** execute the query using the RDF Dataset given in the protocol request.

Note that a service **may** reject a query with HTTP response code 400 if the service does not allow protocol clients to specify the RDF Dataset.

If an RDF Dataset is not specified in either the protocol request or the SPARQL query string, then implementations **may** execute the query against an implementation-defined default RDF dataset.

§ 2.2.5 Accepted Response Formats

Protocol clients **should** use HTTP content negotiation [*RFC9110*] to request response formats that the client can consume. See below for more on potential response formats.

§ 2.2.6 Success Responses

The SPARQL Protocol uses the response status codes defined in HTTP to indicate the success or failure of an operation. Consult the HTTP specification [*RFC9110*] for detailed definitions of each status code. While a protocol service **should** use a 2XX HTTP response code for a successful query, it **may** choose instead to use a 3XX response code as per HTTP.

The response body of a successful query operation with a 2XX response is either:

- a SPARQL Results Document in XML [*SPARQL12-RESULTS-XML*], JSON [*SPARQL12-RESULTS-JSON*], or CSV/TSV [*SPARQL12-RESULTS-CSV-TSV*] format (for SPARQL Query forms SELECT and ASK [*SPARQL12-QUERY*]); or,
- an RDF graph *RDF 1.2 Concepts and Abstract Syntax* serialized, for example, in the *RDF 1.2 XML Syntax*, or an equivalent RDF graph serialization, for SPARQL Query forms DESCRIBE and CONSTRUCT [*SPARQL12-QUERY*]).

The content type of the response to a successful query operation must be the media type defined for the format of the response body.

§ 2.2.7 Failure Responses

The HTTP response codes applicable to an unsuccessful query operation include:

- 400 if the SPARQL query supplied in the request is not a legal sequence of characters in the language defined by the SPARQL grammar; or,
- 500 if the service fails to execute the query. SPARQL Protocol services may also return a 500 response code if they refuse to execute a query. This response does not indicate whether the server may or may not process a subsequent, identical request or requests.

The response body of a failed query request is implementation defined. Implementations **may** use HTTP content negotiation to provide human-readable or machine-processable (or both) information about the failed query request.

A protocol service **may** use other 4XX or 5XX HTTP response codes for other failure conditions, as per HTTP.

§ 2.3 Update Operation

The `update` operation is used to send a SPARQL update request to a service. The update operation **must** be invoked using the HTTP POST method. Client requests for this operation **must** include exactly one SPARQL update request string (parameter name: `update`) and **may** include a version (parameter name: `version`) and zero or more default graph URIs (parameter

name: `using-graph-uri`) and named graph URIs (parameter name: `using-named-graph-uri`). The response to an update request indicates success or failure of the request via HTTP response status code.

	HTTP Method	Query String Parameters	Request Content Type	Request Message Body
update via URL-encoded POST	POST	None	`application/x-www-form-urlencoded`	URL-encoded, ampersand-separated query parameters. `version` (0 or 1) `update` (exactly 1) `using-graph-uri` (0 or more) `using-named-graph-uri` (0 or more)
update via POST directly	POST	`using-graph-uri` (0 or more) `using-named-graph-uri` (0 or more)	`application/sparql-update` (with optional `version` media-type parameter)	Unencoded SPARQL update request string

The `update` request's parameters **must** be sent according to one of these two options:

§ 2.3.1 `update` via POST with URL-encoded parameters

Protocol clients **may** send update protocol requests via the HTTP POST method by URL encoding the parameters. When using this approach, clients **must** URL percent encode all parameters and include them as parameters within the request body via the `application/x-www-form-urlencoded` media type with the name given above. Parameters **must** be separated with the ampersand (&) character. Clients may include the parameters in any order. The content type header of the HTTP request **must** be set to `application/x-www-form-urlencoded`.

§ 2.3.2 `update` via POST directly

Protocol clients **may** send update protocol requests via the HTTP POST method by including the update request directly and unencoded as the HTTP request message body. When using this approach, clients **must** include the SPARQL update request string, unencoded, and nothing else as the message body of the request. Clients **must** set the content type header of the HTTP request to `application/sparql-update`, with an optional `version` media-type parameter. Clients **may** include the optional `using-graph-uri` and `using-named-graph-uri` parameters as HTTP query string parameters in the request URI.

§ 2.3.3 Specifying an RDF Dataset

SPARQL Update requests are executed against a Graph Store, a mutable container of RDF graphs managed by a SPARQL service. The `WHERE` clause of a SPARQL update DELETE/INSERT operation matches against data in an RDF Dataset, which is a subset of the Graph Store. The RDF Dataset for an update operation may be specified either in the operation string itself using the `USING`, `USING NAMED`, and/or `WITH` keywords, or it may be specified via the `using-graph-uri` and `using-named-graph-uri` parameters.

It is an error to supply the `using-graph-uri` or `using-named-graph-uri` parameters when using this protocol to convey a SPARQL 1.2 Update request that contains an operation that uses the `USING`, `USING NAMED`, or `WITH` clause.

A SPARQL Update processor should treat each occurrence of the `using-graph-uri=g` parameter in an update protocol operation as if a `USING <g>` clause were included for every operation in the SPARQL 1.2 Update request. Similarly, a SPARQL Update processor should treat each occurrence of the `using-named-graph-uri=g` parameter in an update protocol operation as if a `USING NAMED <g>` clause were included for every operation in the SPARQL 1.2 Update request.

§ 2.3.4 Success Responses

The SPARQL Protocol uses the response status codes defined in HTTP to indicate the success or failure of an operation. Consult the HTTP specification [*RFC9110*] for detailed definitions of each status code. While a protocol service **should** use a `2XX` HTTP response code for an update

request that is successfully processed, it **may** choose instead to use a `3XX` response code as per HTTP.

The response body of a successful update request is implementation defined. Implementations **may** use HTTP content negotiation to provide both human-readable and machine-processable information about the completed update request.

§ 2.3.5 Failure Responses

The HTTP response code for an unsuccessful update request should be:

- 400 if the SPARQL update request string is not a legal sequence of characters in the language defined by the SPARQL Update grammar; or,
- 500 if the service fails to execute the update request. SPARQL Protocol services may also return a 500 response code if they refuse to execute an update request. This response does not indicate whether the server may or may not process a subsequent, identical request or requests.

The response body of a failed update request is implementation defined. Implementations **may** use HTTP content negotiation to provide human-readable or machine-processable (or both) information about the failed update request.

A protocol service **may** use other 4XX or 5XX HTTP response codes for other failure conditions, as per HTTP.

§ 2.4 Determining the Base IRI

The `BASE` keyword in a SPARQL query or a SPARQL update request string defines the Base IRI used to resolve relative IRIs per *Uniform Resource Identifier (URI): Generic Syntax* section 5.1.1, "Base URI Embedded in Content". The SPARQL Protocol does not dereference query URIs so section 5.1.3 does not apply. Finally, per section 5.1.4, SPARQL Protocol services **must** define their own base URI, which **may** be the service endpoint.

§ 3. Example SPARQL Protocol Requests (informative)

The following HTTP trace examples illustrate invocation of the `query` and `update` operations under several different scenarios. Some example traces are abstracted from complete HTTP traces in these ways:

1. In some examples the string "*EncodedQuery*" represents the URL-encoded string equivalent of the SPARQL query string given in the example; the string "*UnencodedQuery*" represents the exact SPARQL query string given in the example without any encoding.
2. For `query` operation examples, only partial response bodies, containing the query results, are displayed.

§ 3.1 Examples of SPARQL Query

§ 3.1.1 SELECT with service-supplied RDF Dataset

This SPARQL query

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
SELECT ?book ?who
WHERE { ?book dc:creator ?who }
```

is conveyed via HTTP GET to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=PREFIX%20dc%3A%20%3Chttp%3A%2F%2Fpurl.org%2Fd
Host: www.example
User-agent: my-sparql-client/0.1
```

That query against the service-supplied RDF Dataset, executed by that SPARQL query service, returns the following query result:

```
HTTP/1.1 200 OK
Date: Fri, 06 May 2005 20:55:12 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">

 <head>
   <variable name="book"/>
   <variable name="who"/>
 </head>
 <results>
   <result>
     <binding name="book"><uri>http://www.example/book/book5</ur
     <binding name="who"><bnode>r29392923r2922</bnode></binding>
   </result>
...
</sparql>
```

§ 3.1.2 SELECT with simple RDF Dataset

This SPARQL query

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
SELECT ?book ?who
WHERE { ?book dc:creator ?who }
```

is conveyed to the SPARQL query service, `http://www.other.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=PREFIX%20dc%3A%20%3Chttp%3A%2F%2Fpurl.org%2Fd
Host: www.other.example
User-agent: my-sparql-client/0.1
```

That query — against the RDF Dataset identified by the value of the `default-graph-uri` parameter, `http://www.other.example/books` — executed by that SPARQL query service, returns the following query result:

```
HTTP/1.1 200 OK
Date: Fri, 06 May 2005 20:55:12 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
 <head>
   <variable name="book"/>
   <variable name="who"/>
 </head>
...
</sparql>
```

§ 3.1.3 CONSTRUCT with simple RDF dataset and HTTP content negotiation

This SPARQL query

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX myfoaf: <http://www.example/jose/foaf.rdf#>

CONSTRUCT { myfoaf:jose foaf:depiction <http://www.example/jose/jose
            myfoaf:jose foaf:schoolHomepage <http://www.edu.example/
            ?s ?p ?o.}
WHERE { ?s ?p ?o. myfoaf:jose foaf:nick "Jo".
       FILTER ( ! (?s = myfoaf:kendall && ?p = foaf:knows && ?o = my
              && ! ( ?s = myfoaf:julia && ?p = foaf:mbox && ?o = <ma
              && ! ( ?s = myfoaf:julia && ?p = rdf:type && ?o = foaf
}
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
Host: www.example
User-agent: sparql-client/0.1
Accept: text/turtle, application/rdf+xml
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Fri, 06 May 2005 20:55:11 GMT
Server: Apache/1.3.29 (Unix)
Connection: close
Content-Type: text/turtle

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX myfoaf: <http://www.example/jose/foaf.rdf#>

myfoaf:jose foaf:name "Jose Jimeñez";
            foaf:depiction <http://www.example/jose/jose.jpg>;
            foaf:nick "Jo";
...
```

§ 3.1.4 ASK with simple RDF Dataset

This SPARQL query

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
ASK WHERE { ?book dc:creator "J.K. Rowling"}
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
Host: www.example
User-agent: sparql-client/0.1
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Fri, 06 May 2005 20:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
 <head></head>
 <boolean>true</boolean>
</sparql>
```

§ 3.1.5 DESCRIBE with simple RDF Dataset

This SPARQL query

```
PREFIX books: <http://www.example/book/>
DESCRIBE books:book6
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated here:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
Host: www.example
User-agent: sparql-client/0.1
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type:  application/rdf+xml

<?xml version="1.0"?>
<rdf:RDF
    xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
    xmlns:books="http://www.example/book/"
    xmlns:dc="http://purl.org/dc/elements/1.1/" >
  <rdf:Description rdf:about="http://www.example/book/book6">
    <dc:title>Example Book #6 </dc:title>
  </rdf:Description>
</rdf:RDF>
```

§ 3.1.6 SELECT with complex RDF Dataset

This SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>

SELECT ?who ?g ?mbox
WHERE {  ?g dc:publisher ?who .
   GRAPH ?g { ?x foaf:mbox ?mbox }
}
```

is conveyed to the SPARQL query service, http://www.example/sparql/, as illustrated here (with line breaks for legibility):

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
&default-graph-uri=http%3A%2F%2Fwww.example%2Fmorepublishers&nam
&named-graph-uri=http%3A%2F%2Fwww.example%2Ffoaf-bob&named-graph
&named-graph-uri=http%3A%2F%2Fthis.example%2Fjohn%2Ffoaf
Host: www.example
User-agent: sparql-client/0.1
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type:  application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
  <head>
    <variable name="who"/>
    <variable name="g"/>
    <variable name="mbox"/>
  </head>
...
</sparql>
```

§ 3.1.7 SELECT with query-only RDF Dataset

This SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>

SELECT ?who ?g ?mbox
FROM <http://www.example/publishers>
FROM NAMED <http://www.example/alice>
FROM NAMED <http://www.example/bob>
WHERE { ?g dc:publisher ?who .
        GRAPH ?g { ?x foaf:mbox ?mbox }
}
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery HTTP/1.1
Host: www.example
User-agent: sparql-client/0.1
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
...
</sparql>
```

§ 3.1.8 SELECT with ambiguous RDF Dataset

This SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>

SELECT ?who ?g ?mbox
FROM <http://www.example/publishers>
FROM NAMED <http://www.example/john>
FROM NAMED <http://www.example/susan>
WHERE { ?g dc:publisher ?who .
        GRAPH ?g { ?x foaf:mbox ?mbox }
}
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
&named-graph-uri=http%3A%2F%2Fwww.example%2Fbob&named-graph-uri=
Host: www.example
User-agent: sparql-client/0.1
```

This protocol operation contains an ambiguous RDF Dataset: the dataset specified in the query is different than the one specified in the protocol (by way of `default-graph-uri` and `named-graph-uri` parameters). A conformant SPARQL Protocol service must resolve this ambiguity by executing the query against the RDF Dataset specified in the protocol:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
  <head>
    <variable name="who"/>
    <variable name="g"/>
    <variable name="mbox"/>
  </head>
  <results>
    <result>
      <binding name="who">
        <literal>Bob Hacker</literal>
      </binding>
      <binding name="g">
        <uri>http://www.example/bob</uri>
      </binding>
      <binding name="mbox">
        <uri>mailto:bob@oldcorp.example</uri>
      </binding>
    </result>
    <result>
      <binding name="who">
        <literal>Alice Hacker</literal>
      </binding>
      <binding name="g">
        <uri>http://www.example/alice</uri>
      </binding>
      <binding name="mbox">
        <uri>mailto:alice@work.example</uri>
      </binding>
    </result>
  </results>
</sparql>
```

§ 3.1.9 SELECT with malformed query fault

This syntactically invalid SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name
WHERE { ?x foaf:name ?name
ORDER BY ?name }
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fw
Host: www.example
User-agent: sparql-client/0.1
```

With the error response illustrated here:

```
HTTP/1.1 400 Bad Request
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: text/plain; charset=UTF-8

4:syntax error, unexpected ORDER, expecting '}'
```

§ 3.1.10 SELECT with query request refused fault

This SPARQL query

```
PREFIX bio: <http://bio.example/schema/#>
SELECT ?valence
FROM <http://another.example/protein-db.rdf>
WHERE { ?x bio:protein ?valence }
ORDER BY ?valence
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=EncodedQuery&default-graph-uri=http%3A%2F%2Fa
Host: www.example
User-agent: sparql-client/0.1
```

With the error response illustrated here:

```
HTTP/1.1 500 Internal Server Error
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: text/plain

SPARQL Processing Service: Query Request Refused

Your request could not be processed because http://another.examp
could not be retrieved within the time alloted.
```

§ 3.1.11 Long SELECT query using POST with URL encoding

Some SPARQL queries, perhaps machine generated, may be longer than can be reliably conveyed by way of the HTTP GET binding described in 2.1.1 query via GET. In those cases the POST binding described in 2.1.2 query via POST with URL-encoded parameters may be used. This SPARQL query

```
PREFIX : <http://www.w3.org/2002/12/cal/icaltzd#>
PREFIX Chi: <http://www.w3.org/2002/12/cal/test/Chiefs.ics#>
PREFIX New: <http://www.w3.org/2002/12/cal/tzd/America/New_York#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?summary
WHERE {
    {
        Chi:D603E2AC-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-09-08T16:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:09:27Z"^^xsd:dateTime;
         :dtstart "2002-09-08T13:00:00"^^New:tz;
         :summary ?summary;
         :uid "D603E2AC-C1C9-11D6-9446-003065F198AC" .
        }
        UNION
    {
        Chi:D603E90B-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-09-15T16:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:10:19Z"^^xsd:dateTime;
         :dtstart "2002-09-15T13:00:00"^^New:tz;
         :summary ?summary;
         :uid "D603E90B-C1C9-11D6-9446-003065F198AC" .
        }
        UNION
    {
        Chi:D603ED6E-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-09-22T16:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:11:05Z"^^xsd:dateTime;
         :dtstart "2002-09-22T13:00:00"^^New:tz;
         :summary ?summary;
         :uid "D603ED6E-C1C9-11D6-9446-003065F198AC" .
        }
        UNION
    {
        Chi:D603F18C-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-09-29T16:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:15:46Z"^^xsd:dateTime;
         :dtstart "2002-09-29T13:00:00"^^New:tz;
         :summary ?summary;
         :uid "D603F18C-C1C9-11D6-9446-003065F198AC" .
```

```
    }
    UNION
{
    Chi:D603F5B7-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-11-04"^^xsd:date;
     :dtstamp "2002-09-06T03:12:53Z"^^xsd:dateTime;
     :dtstart "2002-11-03"^^xsd:date;
     :summary ?summary;
     :uid "D603F5B7-C1C9-11D6-9446-003065F198AC" .
    }
    UNION
{
    Chi:D603F9D7-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-11-10T20:15:00"^^New:tz;
     :dtstamp "2002-09-06T03:14:12Z"^^xsd:dateTime;
     :dtstart "2002-11-10T17:15:00"^^New:tz;
     :summary ?summary;
     :uid "D603F9D7-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D604022C-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-11-17T17:00:00"^^New:tz;
     :dtstamp "2002-09-06T03:14:51Z"^^xsd:dateTime;
     :dtstart "2002-11-17T14:00:00"^^New:tz;
     :summary ?summary;
     :uid "D604022C-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D604065C-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-10-06T19:05:00"^^New:tz;
     :dtstamp "2002-09-06T03:16:54Z"^^xsd:dateTime;
     :dtstart "2002-10-06T16:05:00"^^New:tz;
     :summary ?summary;
     :uid "D604065C-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D6040A7E-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-10-13T19:15:00"^^New:tz;
     :dtstamp "2002-09-06T03:17:51Z"^^xsd:dateTime;
```

```
     :dtstart "2002-10-13T16:15:00"^^New:tz;
     :summary ?summary;
     :uid "D6040A7E-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D6040E96-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-10-20T16:00:00"^^New:tz;
     :dtstamp "2002-09-06T03:18:32Z"^^xsd:dateTime;
     :dtstart "2002-10-20T13:00:00"^^New:tz;
     :summary ?summary;
     :uid "D6040E96-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D6041270-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-10-27T17:00:00"^^New:tz;
     :dtstamp "2002-09-06T03:19:15Z"^^xsd:dateTime;
     :dtstart "2002-10-27T14:00:00"^^New:tz;
     :summary ?summary;
     :uid "D6041270-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D6041673-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-11-24T20:05:00"^^New:tz;
     :dtstamp "2002-09-06T03:22:09Z"^^xsd:dateTime;
     :dtstart "2002-11-24T17:05:00"^^New:tz;
     :summary ?summary;
     :uid "D6041673-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
    Chi:D6041A73-C1C9-11D6-9446-003065F198AC     a :Vevent;
     :dtend "2002-12-01T17:00:00"^^New:tz;
     :dtstamp "2002-09-06T03:22:52Z"^^xsd:dateTime;
     :dtstart "2002-12-01T14:00:00"^^New:tz;
     :summary ?summary;
     :uid "D6041A73-C1C9-11D6-9446-003065F198AC" .
}
    UNION
{
```

```
        Chi:D60421EF-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-12-08T17:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:24:04Z"^^xsd:dateTime;
         :dtstart "2002-12-08T14:00:00"^^New:tz;
         :summary ?summary;
         :uid "D60421EF-C1C9-11D6-9446-003065F198AC" .
    }
        UNION
    {
        Chi:D6042660-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-12-15T20:05:00"^^New:tz;
         :dtstamp "2002-09-06T03:25:03Z"^^xsd:dateTime;
         :dtstart "2002-12-15T17:05:00"^^New:tz;
         :summary ?summary;
         :uid "D6042660-C1C9-11D6-9446-003065F198AC" .
    }
        UNION
    {
        Chi:D6042A93-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-12-22T17:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:25:47Z"^^xsd:dateTime;
         :dtstart "2002-12-22T14:00:00"^^New:tz;
         :summary ?summary;
         :uid "D6042A93-C1C9-11D6-9446-003065F198AC" .
    }
        UNION
    {
        Chi:D6042EDF-C1C9-11D6-9446-003065F198AC     a :Vevent;
         :dtend "2002-12-28T21:00:00"^^New:tz;
         :dtstamp "2002-09-06T03:26:51Z"^^xsd:dateTime;
         :dtstart "2002-12-28T18:00:00"^^New:tz;
         :summary ?summary;
         :uid "D6042EDF-C1C9-11D6-9446-003065F198AC" .
    }
}
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
POST /sparql/ HTTP/1.1
Host: www.example
User-agent: sparql-client/0.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 9461

query=EncodedQuery&default-graph-uri=http%3A%2F%2Fanother.exampl
```

With the response illustrated here:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix) PHP/4.3.4 DAV/1.0.3
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
    <head>
        <variable name="summary"/>
    </head>
    <results>
        <result>
            <binding name="summary">
                <literal>Chiefs vs. Cleveland @ Cleveland Stadiu
            </binding>
        </result>
        <result>
            <binding name="summary">
                <literal>Chiefs vs. Jacksonville @ Arrowhead Sta
            </binding>
        </result>
        <result>
            <binding name="summary">
                <literal>Chiefs vs. New England @ Gillette Stadi
            </binding>
        </result>
        ...
        <result>
            <binding name="summary">
                <literal>Chiefs vs. Oakland @ Network Associates
            </binding>
        </result>
    </results>
</sparql>
```

§ 3.1.12 Long SELECT query using direct POST

SPARQL queries may also be POSTed directly without URL encoding, as described in 2.1.3 query via POST directly. The same query used in the previous example is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
POST /sparql/?default-graph-uri=http%3A%2F%2Fanother.example%2Fc
Host: www.example
User-agent: sparql-client/0.1
Content-Type: application/sparql-query

UnencodedQuery
```

With the same response as in the previous example.

§ 3.1.13 SELECT with internationalization

SPARQL queries may include internationalized characters or character sets. This SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX 食: <http://www.w3.org/2001/sw/DataAccess/tests/data/i18n/kan
SELECT ?name ?food
WHERE { [ foaf:name ?name ; 食:食べる ?food ] . }
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`, as illustrated in this HTTP trace:

```
GET /sparql/?query=PREFIX%20foaf%3A%20%3Chttp%3A%2F%2Fxmlns.com%
Host: www.example
User-agent: sparql-client/0.1
```

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix)
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
...
</sparql>
```

§ 3.1.14 SELECT with reified triple patterns

SPARQL queries can target reified triples. This SPARQL query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX ex: <http://example.org/>
SELECT ?name ?person
WHERE { << _:a foaf:name ?name >> ex:statedBy ?person . }
```

is conveyed to the SPARQL query service, `http://www.example/sparql/`. Since the query is using SPARQL 1.2 syntax, and the query does not announce the 1.2 version syntactically, the GET request is issued using the `version=1.2` parameter. This is illustrated in this HTTP trace:

```
GET /sparql/?query=PREFIX%20foaf%3A%20%3Chttp%3A%2F%2Fxmlns.com%
Host: www.example
User-agent: sparql-client/0.1
```

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix)
Connection: close
Content-Type: application/sparql-results+xml

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
...
</sparql>
```

§ 3.1.15 SELECT with 1.2 query and 1.2 response using direct POST

The query below uses the `isTRIPLE` function. Since this is a SPARQL 1.2 feature, it announces its version syntactically:

```
VERSION "1.2"
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX ex: <http://example.org/>
SELECT ?statement ?person
WHERE { ?statement ex:statedBy ?person . FILTER (isTRIPLE(?statement
```

While the query announces its version syntactically, the SPARQL client decides to send the direct POST request using the `version=1.2` media-type, which is allowed but not required if the version is already announced syntactically. This is illustrated in this HTTP trace:

```
POST /sparql HTTP/1.1
Host: www.example
User-agent: sparql-client/0.1
Content-Type: application/sparql-query; version=1.2

UnencodedQuery
```

Since the response will contain RDF 1.2 triple terms, the server responds with a `version=1.2` media type parameter:

```
HTTP/1.1 200 OK
Date: Wed, 03 Aug 2005 12:48:25 GMT
Server: Apache/1.3.29 (Unix)
Connection: close
Content-Type: application/sparql-results+xml; version=1.2

<?xml version="1.0"?>
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
...
</sparql>
```

§ 3.2 Examples of SPARQL Update

§ 3.2.1 UPDATE using URL-encoded parameters

An example request, which is a serialisation of a request sent to `http://localhost:8888/test` for the query `INSERT DATA { <a> <p> <b> }` is shown below using the URL-encoded parameter form.

```
POST /test HTTP/1.1
Host: localhost:8888
Accept: text/plain
Content-Length: 62
Content-Type: application/x-www-form-urlencoded

update=INSERT%20DATA%20%7B%20%3Ca%3E%20%3Cp%3E%20%3Cb%3E%20%7D
```

§ 3.2.2 UPDATE using POST directly

Update requests may be sent as a POST request with a Content-Type of `application/sparql-update`:

```
POST /test HTTP/1.1
Host: localhost:8888
Accept: */*
Content-Type: application/sparql-update
Content-Length: 27

INSERT DATA { <a> <p> <b> }
```

§ 3.2.3 UPDATE specifying dataset and using POST directly

A dataset for an update request may be specified using the `using-graph-uri` and `using-named-graph-uri` parameters. The serialisation of an example request sent to `http://localhost:8888/test` and specifying a dataset with default graph `http://localhost:8888/people` is shown below.

```
POST /test?using-graph-uri=http%3A%2F%2Flocalhost%3A8888%2Fpeopl
Host: localhost:8888
Accept: */*
Content-Type: application/sparql-update
Content-Length: 136

PREFIX foaf: <http://xmlns.com/foaf/0.1/>
DELETE { ?person ?property ?value }
WHERE { ?person ?property ?value ; foaf:givenName 'Fred' }
```

§ 3.2.4 Multi-operation UPDATE using URL-encoded parameters

A sequence of multiple operations may be included in a single request, separated by a ';' (semicolon). The serialisation of an example request sent to `http://localhost:8888/test` for the query

```
DELETE DATA { <a> <p> <old> } ;
INSERT DATA { <a> <p> <new> }
```

is shown below using the URL-encoded parameter form.

```
POST /test HTTP/1.1
Host: localhost:8888
Accept: */*
Content-Type: application/x-www-form-urlencoded
Content-Length: 130

update=DELETE%20DATA%20%7B%20%3Ca%3E%20%3Cp%3E%20%3Cold%3E%20%7D
```

§ 3.2.5 Multi-operation UPDATE specifying dataset and using URL-encoded parameters

When POSTing an update request with URL-encoded parameters, the dataset parameters `using-graph-uri` and `using-named-graph-uri` are specified in the POST body with the serialized request. The serialisation of an example request sent to `http://localhost:8888/test` for the query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
INSERT { GRAPH <http://localhost:8888/people> { ?person ?property ?v
WHERE { GRAPH ?g { ?person ?property ?value ; foaf:givenName 'Fred'
```

and specifying a dataset with the named graphs `http://localhost:8888/alice/foaf.rdf` and `http://localhost:8888/eve/foaf.rdf` is shown below.

```
POST /test HTTP/1.1
Host: localhost:8888
Accept: */*
Content-Type: application/x-www-form-urlencoded
Content-Length: 130

using-named-graph-uri=http%3A%2F%2Flocalhost%3A8888%2Falice%2Ffo
```

§ 3.2.6 Multi-operation UPDATE specifying dataset and using POST directly

The serialisation of an example request sent to `http://localhost:8888/test` and specifying a dataset with the named graphs `http://localhost:8888/alice/foaf.rdf` and `http://localhost:8888/eve/foaf.rdf` is shown below.

```
POST /test?using-named-graph-uri=http%3A%2F%2Flocalhost%3A8888%2
Host: localhost:8888
Accept: */*
Content-Type: application/sparql-update
Content-Length: 190

PREFIX foaf: <http://xmlns.com/foaf/0.1/>
INSERT { GRAPH <http://localhost:8888/people> { ?person ?propert
WHERE { GRAPH ?g { ?person ?property ?value ; foaf:givenName 'Fr
```

§ 4. Policy Considerations

§ 4.1 Security

There are at least two possible sources of denial-of-service attacks against SPARQL protocol services. First, under-constrained queries can result in very large numbers of results, which may require large expenditures of computing resources to process, assemble, or return. Another possible source are queries containing very complex — either because of resource size, the number of resources to be retrieved, or a combination of size and number — RDF Dataset descriptions, which the service may be unable to assemble without significant expenditure of resources, including bandwidth, CPU, or secondary storage. In some cases such expenditures may effectively constitute a denial-of-service attack. A SPARQL protocol service **may** place restrictions on the resources that it retrieves or on the rate at which external resources are

retrieved. There may be other sources of denial-of-service attacks against SPARQL query processing services.

Since a SPARQL protocol service may make HTTP requests of other origin servers on behalf of its clients, it may be used as a vector of attacks against other sites or services. Thus, SPARQL protocol services may effectively act as proxies for third-party clients. Such services **may** place restrictions on the resources that they retrieve or on the rate at which external resources can be retrieved. SPARQL protocol services **may** log client requests in such a way as to facilitate tracing them with regard to third-party origin servers or services.

SPARQL protocol services **may** choose to detect these and other costly, or otherwise unsafe, queries, impose time or memory limits on queries, or impose other restrictions to reduce the service's (and other service's) vulnerability to denial-of-service attacks. They also **may** refuse to process such query requests.

SPARQL protocol services may remove, insert, and change underlying data via the update operation. To protect against malicious or destructive updates, implementations may choose not to implement the update operation. Alternatively, implementations may choose to use HTTP authentication mechanisms or other implementation-defined mechanisms to prevent unauthorized invocations of the update operation.

Different IRIs may have the same appearance. Characters in different scripts may look similar (a Cyrillic "o" may appear similar to a Latin "o"). A character followed by combining characters may have the same visual representation as another character (LATIN SMALL LETTER E followed by COMBINING ACUTE ACCENT has the same visual representation as LATIN SMALL LETTER E WITH ACUTE). Users of SPARQL must take care to construct queries with IRIs that match the IRIs in the data. Further information about matching of similar characters can be found in *Unicode Security Considerations* and *Internationalized Resource Identifiers (IRIs)* Section 8.

§ 5. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key word *MAY* in this document is to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, it appears in all capitals, as shown here.

§ 5.1 Conformance

The status of the parts of SPARQL 1.2 Protocol (this document) is as follows:

- Section 1 Introduction: **normative**
- Section 2 SPARQL Protocol Operations: **normative**
- Section 3: Example SPARQL Protocol Requests: **informative**
- Section 4: Policy Considerations: **normative**
- Section 5: Conformance: **normative**
- Section 6: Changes Since Previous Recommendation: **informative**
- Section A.1: Normative References: **normative**
- Section A.2: Other References: **informative**

A conformant SPARQL Protocol service:

1. **must** implement either the `query` operation or the `update` operation in the way described in this document ("SPARQL 1.2 Protocol");
2. **may** implement both the `query` and `update` operations;
3. **must** be consistent with the normative constraints (indicated by RFC2119 keywords [*RFC2119*]) described in 4. Policy Considerations.

§ A. Changes between SPARQL 1.1 Protocol and SPARQL 1.2 Protocol

This section is non-normative.

This specification extends and updates the SPARQL 1.1 Protocol of March, 2013. The significant changes are:

- Add informative example of reified triples
- Improve display on mobile phones
- Add explicit references to RFCs
- Use PREFIX instead of @prefix in all Turtle examples

B. Changes between SPARQL 1.0 Protocol and SPARQL 1.1 Protocol

This section is non-normative.

This specification extends and updates the SPARQL Protocol for RDF of January, 2008. The significant changes are:

- Remove the WSDL definition of the protocol in favor of an HTTP-based protocol
- Define an Update operation for issuing SPARQL Update requests
- Updated conformance criteria to accommodate the update operation
- Relaxed the requirements on specific HTTP response codes to allow for other codes as long as they align with HTTP semantics
- Added a variant of the query operation that directly posts a query string in the body of a POST request

C. Privacy Considerations

TODO

§ D. Security Considerations

TODO

§ E. Internationalization Considerations

TODO

§ F. Index

§ F.1 Terms defined by this specification

§ F.2 Terms defined by reference

[*RDF12-CONCEPTS*] defines the following:

RDF 1.2 Concepts and Abstract Syntax

[*RFC2119*] defines the following:

RFC2119

[*RFC3986*] defines the following:

percent encode

percent encode

query parameter

[*RFC9110*] defines the following:

content negotiation

HTTP specification

Section 3.3. Connections, Clients, and Servers

[*SPARQL12-QUERY*] defines the following:

ASK

CONSTRUCT

DESCRIBE

query form

SELECT

SPARQL 1.2 Query Language

version directive

[*SPARQL12-RESULTS-CSV-TSV*] defines the following:

CSV/TSV

[*SPARQL12-RESULTS-JSON*] defines the following:

JSON

[*SPARQL12-RESULTS-XML*] defines the following:

XML

[*SPARQL12-UPDATE*] defines the following:

DELETE/INSERT operation

§ G. References

§ G.1 Normative references

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Syntax. Olaf Hartig; Pierre-Antoine Champin; Gregg Kellogg; Andy Seaborne. W3C. 7 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 12 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 13 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]

Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[RFC9110]

HTTP Semantics. R. Fielding, Ed.; M. Nottingham, Ed.; J. Reschke, Ed. IETF. June 2022. Internet Standard. URL: https://httpwg.org/specs/rfc9110.html

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 8 May 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[UTR36]

Unicode Security Considerations. Mark Davis; Michel Suignard. Unicode Consortium. 19 September 2014. Unicode Technical Report #36. URL: https://www.unicode.org/reports/tr36/tr36-15.html

§ G.2 Informative references

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 5 May 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

↥